

Load Balancer in Go for the Group Chat Application

A reverse proxy on Sys1 with round robin scheduling and health checks, running in front of the group chat back end from the previous assignment on Sys2, Sys3 and Sys4.

STUDENT	ROLL NUMBER	SYSTEMS	DATE
Akshat Gupta	12340160	stu68_sys1 to sys4	24 August 2026

LIVE APPLICATION URL

<https://10.1.75.53:3229>

This is the group chat application from the previous assignment, served through the load balancer on Sys1. Every request is passed to one of the three back ends on Sys2, Sys3 and Sys4. The certificate is self signed, so the browser shows a warning on the first visit. Click Advanced and then Proceed. HTTPS is needed because the chat client signs its login with WebCrypto, and browsers only allow WebCrypto on a secure origin.

1. Assigned systems

Four containers run on one host and share a Docker bridge network. Each has 512 MB of RAM and one vCPU. That limit is why a single back end fills up so quickly.

SYSTEM	ROLE	INTERNAL IP	APP PORT	SSH	PUBLIC ADDRESS
stu68_sys1	Load balancer and MongoDB	172.17.0.30	3000	-p 2229	https://10.1.75.53:3229
stu68_sys2	Back end chat-1	172.17.0.31	4000	-p 2230	http://10.1.75.53:4230
stu68_sys3	Back end chat-2	172.17.0.32	4000	-p 2231	http://10.1.75.53:4231
stu68_sys4	Back end chat-3	172.17.0.33	4000	-p 2232	http://10.1.75.53:4232

SSH goes to student@10.1.75.53 on the port shown. Application ports follow the lab rule that an internal port is published on a host port taken from the SSH port. The load balancer listens on port 3000 inside Sys1, so it is reachable from outside on port 3229. This is why no SSH tunnel was needed.

```
### deployment inventory, 2026-08-24 15:46 UTC ###
all four containers: 512 MB RAM, 1 vCPU (cgroup limits), Ubuntu 24.04.4 LTS

--- stu68_sys1 172.17.0.30 ---
listening : *:3000 0.0.0.0:27017
pid 80946 239 MB /home/student/mongodb/bin/mongod --dbpath /home/student/mongo-data
pid 89206 23 MB /home/student/lb/lb -listen :3000

--- stu68_sys2 172.17.0.31 ---
listening : *:4000
pid 20079 90 MB /home/student/node/bin/node --env-file-if-exists=.env server.js

--- stu68_sys3 172.17.0.32 ---
listening : *:4000
pid 20208 84 MB /home/student/node/bin/node --env-file-if-exists=.env server.js

--- stu68_sys4 172.17.0.33 ---
listening : *:4000
pid 19617 96 MB /home/student/node/bin/node --env-file-if-exists=.env server.js
```

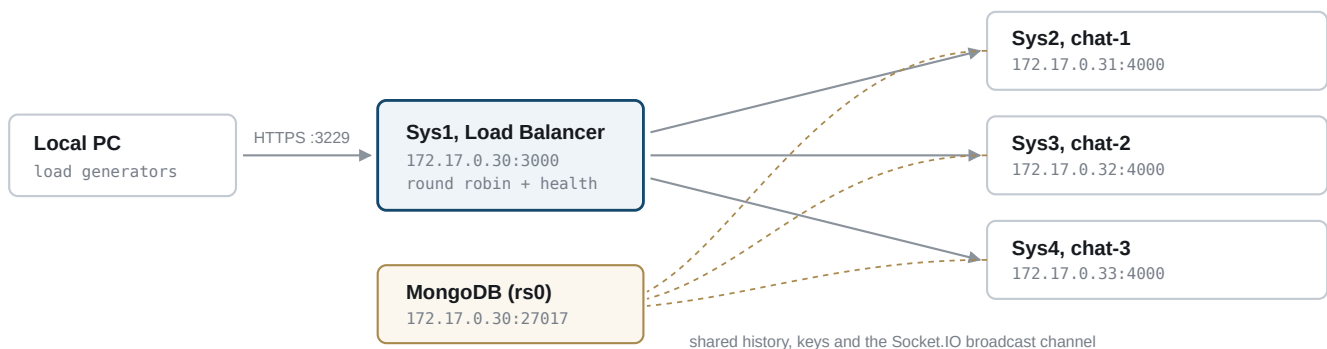
Figure 1. The four systems as deployed. Sys1 runs mongod and the load balancer. Sys2, Sys3 and Sys4 each run one Node process serving the chat back end.

2. What was built

The back end for this assignment is the group chat application from the previous assignment. It is the persistent encrypted chat from `csd-group-chat`, running as three copies instead of the simple HTTP server used in the tutorial slides. A stateless server would work behind a round robin proxy without changes. A chat server does not, so three things were needed.

1. **One shared database.** All three copies use the same MongoDB on Sys1. Chat history and the stored public keys are therefore the same wherever a client lands.
2. **A shared broadcast channel.** Socket.IO keeps connections in the memory of one process. A message sent to the copy on Sys2 would never reach a client on Sys4. The official MongoDB adapter gives the three processes a channel to publish broadcasts on.
3. **Sticky sessions in the load balancer.** The Socket.IO handshake is a set of HTTP polls that only one process can answer. Round robin breaks it, so the load balancer keeps a client on one back end using a cookie.

COMPONENT	WHERE IT RUNS	IMPLEMENTATION
Load balancer	Sys1, port 3000	Go, <code>httputil.ReverseProxy</code> , round robin with health checks, sticky cookie and TLS
Chat back end, three copies	Sys2, Sys3, Sys4, port 4000	Node 24 running the group chat server .js with the Socket.IO MongoDB adapter
Database	Sys1, port 27017	MongoDB 8.0 as a single node replica set
Load generators	Local PC	Go worker pool for plain HTTP, Node worker pool for real chat sessions



3. Load balancer code (Sys1)

The design follows the tutorial. There is a list of backend structs, each holding a reverse proxy. A mutex protects the round robin index. A background loop checks health. The endpoints `/lb/status`, `/lb/metrics` and `/lb/reset` allow the experiments to be run again without restarting the process. Four things were added for this deployment.

ADDITION	REASON
Sticky cookie lb_backend	The Socket.IO handshake spans several requests and only one process can answer them. A client with no cookie is round robin and given one. A client with a cookie keeps its back end until that back end dies, and then falls back to round robin.
Connection pool MaxIdleConnsPerHost: 256	The default transport keeps only two idle connections per host. Above a low rate almost every request opened a new TCP connection. That held the proxy at 443 req/s while the back ends served 1244 req/s on their own. Fixing it raised the proxy to 674 req/s on the same test.
TLS -tls-cert and -tls-key	The chat client signs its login with WebCrypto. Browsers only allow WebCrypto on a secure origin. Over plain HTTP the app loads but nobody can log in.
Three failed checks before marking down	One slow reply is not a dead server. A back end is only removed after three failed checks in a row.

Round robin is still what the experiments measure. Both load generators run without a cookie jar, so all of their requests take the round robin path. Stickiness only affects browsers and Socket.IO clients. This is why the split across back ends in Section 7 comes out even.

```
// Reverse proxy for Sys1, in front of the chat servers on Sys2, Sys3 and Sys4.
// Requests without the sticky cookie are round robin. Requests with it stay
// on the backend named in the cookie.
package main

import (
    "bufio"
    "encoding/json"
    "flag"
    "log"
    "net"
    "net/http"
    "net/http/httputil"
    "net/url"
    "sort"
    "strconv"
    "strings"
    "sync"
    "time"
)

type Backend struct {
    ID      int
    URL     *url.URL
    Proxy   *httputil.ReverseProxy
    Alive   bool
    Count   int // requests sent here by round robin
    Sticky  int // requests sent here because of the cookie
    fails   int // failed health checks in a row
}

var (
    mu      sync.Mutex
    backends []*Backend
    current int // round robin position
    total   int
    success int
    failed  int
    latencies []time.Duration
    sticky   bool
)

const stickyCookie = "lb_backend"

// picks the next alive backend, round robin
func nextBackend() *Backend {
    mu.Lock()
    defer mu.Unlock()

    for i := 0; i < len(backends); i++ {
        b := backends[i]
        current = (current + 1) % len(backends)
        if b.Alive {
            b.Count++
        }
    }
}
```

```

        return b
    }
}
return nil // everything is down
}

// Returns nil if the cookie is missing or its backend is down, so the caller
// falls back to round robin.
func stickyBackend(r *http.Request) *Backend {
    if !sticky {
        return nil
    }

    c, err := r.Cookie(stickyCookie)
    if err != nil {
        return nil
    }

    id, err := strconv.Atoi(c.Value)
    if err != nil || id < 0 || id >= len(backends) {
        return nil
    }

    mu.Lock()
    defer mu.Unlock()

    b := backends[id]
    if !b.Alive {
        return nil
    }

    b.Sticky++
    return b
}

func record(d time.Duration, ok bool) {
    mu.Lock()
    total++
    if ok {
        success++
        latencies = append(latencies, d)
    } else {
        failed++
    }
    mu.Unlock()
}

// wraps ResponseWriter to capture the status code
type recorder struct {
    http.ResponseWriter
    code int
}

func (r *recorder) WriteHeader(c int) {
    r.code = c
    r.ResponseWriter.WriteHeader(c)
}

// lets websocket upgrades through
func (r *recorder) Hijack() (net.Conn, *bufio.ReadWriter, error) {
    return r.ResponseWriter.(http.Hijacker).Hijack()
}

func proxyHandler(w http.ResponseWriter, r *http.Request) {
    start := time.Now()

    b := stickyBackend(r)
    pinned := b != nil

    if b == nil {
        b = nextBackend()
    }

    if b == nil {
        record(time.Since(start), false)
        http.Error(w, "no backend available", http.StatusServiceUnavailable)
        return
    }

    // Must be set before ServeHTTP. ReverseProxy overwrites this header map.
    if sticky && !pinned {
        http.SetCookie(w, &http.Cookie{
            Name: stickyCookie,

```

```

        Value: strconv.Itoa(b.ID),
        Path:  "/",
        MaxAge: 3600,
    })
}

rec := &recorder{w, http.StatusOK}
b.Proxy.ServeHTTP(rec, r)

record(time.Since(start), rec.code < 500)
}

// marks a backend down after three failed checks in a row
func healthCheck(path string, every time.Duration) {
    client := &http.Client{
        Timeout: 5 * time.Second,
        Transport: &http.Transport{DisableKeepAlives: true},
    }

    for {
        for _, b := range backends {
            resp, err := client.Get(b.URL.String() + path)
            good := err == nil && resp.StatusCode < 400
            if resp != nil {
                resp.Body.Close()
            }

            mu.Lock()
            if good {
                b.fails = 0
                if !b.Alive {
                    log.Printf("%s is back up", b.URL)
                    b.Alive = true
                }
            } else {
                b.fails++
                if b.fails >= 3 && b.Alive {
                    log.Printf("%s marked down after %d failed checks: %v", b.URL, b.fails, err)
                    b.Alive = false
                }
            }
            mu.Unlock()
        }
        time.Sleep(every)
    }
}

func percentile(sorted []time.Duration, p float64) float64 {
    if len(sorted) == 0 {
        return 0
    }
    i := int(p / 100 * float64(len(sorted)))
    if i >= len(sorted) {
        i = len(sorted) - 1
    }
    return float64(sorted[i].Microseconds()) / 1000
}

func statusHandler(w http.ResponseWriter, r *http.Request) {
    mu.Lock()
    defer mu.Unlock()

    out := []map[string]interface{}{}
    for _, b := range backends {
        out = append(out, map[string]interface{}{
            "id":      b.ID,
            "url":     b.URL.String(),
            "alive":   b.Alive,
            "served":  b.Count,
            "sticky":  b.Sticky,
        })
    }
    writeJSON(w, map[string]interface{}{
        "sticky_sessions": sticky,
        "backends":        out,
    })
}

func metricsHandler(w http.ResponseWriter, r *http.Request) {
    mu.Lock()
    defer mu.Unlock()

    sorted := make([]time.Duration, len(latencies))

```

```

copy(sorted, latencies)
sort.Slice(sorted, func(i, j int) bool { return sorted[i] < sorted[j] })

perBackend := map[string]int{}
for _, b := range backends {
    perBackend[b.URL.Host] = b.Count
}

writeJSON(w, map[string]interface{}{
    "total":      total,
    "success":    success,
    "failed":     failed,
    "p50_ms":     percentile(sorted, 50),
    "p95_ms":     percentile(sorted, 95),
    "p99_ms":     percentile(sorted, 99),
    "per_backend": perBackend,
})
}

// clears the counters between experiments
func resetHandler(w http.ResponseWriter, r *http.Request) {
    mu.Lock()
    total, success, failed = 0, 0, 0
    latencies = nil
    for _, b := range backends {
        b.Count = 0
        b.Sticky = 0
    }
    mu.Unlock()
    w.Write([]byte("reset done\n"))
}

func writeJSON(w http.ResponseWriter, v interface{}) {
    w.Header().Set("Content-Type", "application/json")
    e := json.NewEncoder(w)
    e.SetIndent("", " ")
    e.Encode(v)
}

func main() {
    listen := flag.String("listen", ":8080", "port to listen on")
    list := flag.String("backends", "", "comma separated backend urls")
    healthPath := flag.String("health-path", "/health", "path to check on backends")
    useSticky := flag.Bool("sticky", true, "pin a client to one backend with a cookie")
    // HTTPS is needed because the chat client uses WebCrypto to log in.
    tlsCert := flag.String("tls-cert", "", "PEM certificate, turns on HTTPS with -tls-key")
    tlsKey := flag.String("tls-key", "", "PEM private key")
    flag.Parse()

    if *list == "" {
        log.Fatal("give me some backends with -backends")
    }

    sticky = *useSticky

    for i, raw := range strings.Split(*list, ",") {
        u, err := url.Parse(strings.TrimSpace(raw))
        if err != nil {
            log.Fatalf("bad url %s: %v", raw, err)
        }

        b := &Backend{ID: i, URL: u, Alive: true}
        b.Proxy = httputil.NewSingleHostReverseProxy(u)

        // The default transport keeps two idle connections per host, which
        // capped the proxy at 443 req/s.
        b.Proxy.Transport = &http.Transport{
            MaxIdleConns:    512,
            MaxIdleConnsPerHost: 256,
            IdleConnTimeout:   90 * time.Second,
            DialContext: (&net.Dialer{
                Timeout:   3 * time.Second,
                KeepAlive: 30 * time.Second,
            }).DialContext,
        }

        // names the backend that answered
        b.Proxy.ModifyResponse = func(resp *http.Response) error {
            resp.Header.Set("X-LB-Backend", u.Host)
            return nil
        }

        // count a refused connection as a failed check

```

```

    b.Proxy.ErrorHandler = func(w http.ResponseWriter, r *http.Request, err error) {
        log.Printf("error from %s: %v", u, err)
        mu.Lock()
        b.fails++
        if b.fails >= 3 {
            b.Alive = false
        }
        mu.Unlock()
        http.Error(w, "backend error", http.StatusBadGateway)
    }

    backends = append(backends, b)
    log.Printf("added backend %s", u)
}

go healthCheck(*healthPath, 2*time.Second)

http.HandleFunc("/lb/health", func(w http.ResponseWriter, r *http.Request) {
    w.Write([]byte("ok\n"))
})
http.HandleFunc("/lb/status", statusHandler)
http.HandleFunc("/lb/metrics", metricsHandler)
http.HandleFunc("/lb/reset", resetHandler)
http.HandleFunc("/", proxyHandler)

if *tlsCert != "" && *tlsKey != "" {
    log.Printf("load balancer running on %s with %d backends (sticky=%v, TLS)", *listen, len(backends), sticky)
    log.Fatal(http.ListenAndServeTLS(*listen, *tlsCert, *tlsKey, nil))
}

log.Printf("load balancer running on %s with %d backends (sticky=%v)", *listen, len(backends), sticky)
log.Fatal(http.ListenAndServe(*listen, nil))
}

```

Figure 2. lb/main.go, the load balancer as built and run on Sys1.

4. Back end deployment (Sys2, Sys3, Sys4)

Each system runs the group chat server with the patch below and nothing else changed. Node 24 was installed from the official tarball into ~/node, because there is no passwordless sudo on these containers and nothing could be installed system wide.

The patch makes two changes. The first attaches the Socket.IO MongoDB adapter so a broadcast from one copy reaches clients on the other two. The second adds an X-Backend header naming the copy, so a response can be traced to one machine. The encryption, the signatures, the stored public keys and the rate limiter were all left alone.

```

--- server/server.js (previous assignment)
+++ server/server.js (this deployment)
@@ -9,6 +9,7 @@
const { createHealthRouter } = require('./src/routes/health');
const { loadKey } = require('./src/crypto/messageCipher');
const db = require('./src/db');
+const { createAdapter } = require('@socket.io/mongo-adapter');

const app = express();
const httpServer = createServer(app);
@@ -18,11 +19,24 @@
// that port lets both run at once.
const PORT = process.env.PORT || 4000;

+// Which of the three replicas this is.
+const INSTANCE = process.env.INSTANCE_NAME || 'backend';
+
+// Socket.IO holds its connections in one process, so the three replicas pass
+// broadcasts to each other through this capped collection instead.
+const ADAPTER_COLLECTION = 'socket.io-adapter-events';
+
const presence = new Presence();
const messageRateLimiter = new RateLimiter();

+// Tags every response with the replica that served it.
+app.use((req, res, next) => {
+  res.setHeader('X-Backend', INSTANCE);
+  next();
+});
+

```

```

    app.use(express.static(path.join(__dirname, '..', 'client', 'dist')));
    -app.use(createHealthRouter(presence));
    +app.use(createHealthRouter(presence, INSTANCE));

    // Connect to the database before listening. If we started listening first, a
    // client could send a message before the database was ready and that message
    @@ -33,15 +47,28 @@
    // people are already in the room and that message is lost.
    loadKey();

    - await db.connect();
    + const database = await db.connect();
    console.log('Connected to MongoDB');

    + // Safe to repeat, a second call just reports that it already exists.
    + try {
    +   await database.createCollection(ADAPTER_COLLECTION, {
    +     capped: true,
    +     size: 1e6,
    +   });
    + } catch (err) {
    +   if (err.codeName !== 'NamespaceExists') throw err;
    + }
    +
    + io.adapter(createAdapter(database.collection(ADAPTER_COLLECTION)));
    + console.log(`Socket.IO adapter attached (${INSTANCE})`);
    +
    io.on('connection', (socket) => {
      registerSocketHandlers(io, socket, { presence, messageRateLimiter });
    });

    httpServer.listen(PORT, () => {
    -   console.log(`Chat server running on http://localhost:${PORT}`);
    +   console.log(`Chat server ${INSTANCE} running on http://0.0.0.0:${PORT}`);
    });
  }

```

Figure 3. The full diff against the previous assignment code.

MongoDB runs as a single node replica set because the adapter reads a capped collection using a change stream, and change streams need an oplog that a standalone mongod does not have. The same database also holds the chat history and the public keys that all three copies share.

```

$ ssh stu68_sys2 'bash ~/start-chat.sh chat-1'
UP after 1s
{"status":"ok","backend":"chat-1","uptimeSeconds":1,"onlineUsers":0}

$ for h in sys2 sys3 sys4; do ssh $h 'cat ~/logs/chat.log'; done
--- stu68_sys2 ---
Connected to MongoDB
Socket.IO adapter attached (chat-1)
Chat server chat-1 running on http://0.0.0.0:4000
--- stu68_sys3 ---
Connected to MongoDB
Socket.IO adapter attached (chat-2)
Chat server chat-2 running on http://0.0.0.0:4000
--- stu68_sys4 ---
Connected to MongoDB
Socket.IO adapter attached (chat-3)
Chat server chat-3 running on http://0.0.0.0:4000

$ ssh stu68_sys1 '~/mongodb/bin/mongod ... --replSet rs0' # replica set status
set: rs0 members: 172.17.0.30:27017 = PRIMARY
messages stored: 121
a stored record is ciphertext, not text:
  senderId   : aarav
  ciphertext  : ba/SPIBscbGAR0c6R0e94Da019wUxQi1JWx4eyLufF+L...
  signature   : bfpTMhCMnmtQLEB1wusTeWM9tLzcPqVHZuBTefCKvQJ...

```

Figure 4. Starting the three copies and checking that each one attached the adapter. The last part shows that stored messages are ciphertext and not readable text.

5. Load generators (local system)

Two load generators run on the local PC. Both are worker pools over a fixed number of requests and both write the same JSON and CSV. They exist because they measure different things.

GENERATOR	ONE REQUEST IS	WHAT IT MEASURES
HTTP loadgen_main.go	GET /health	The path itself. The back end answers from memory in microseconds, so this measures the proxy and the network rather than the servers.
Chat chatload_main.js	A full chat session opening. It connects over WebSocket, signs the server challenge with ECDSA, logs in and waits for the chat history.	The back ends. Before replying, the server decrypts and verifies the signature on all 100 stored messages. This is real CPU work.

The chat generator uses the websocket transport only. A websocket connection is a single request, so each session lands on one back end and the split stays even without needing the sticky cookie.

The room holds 120 seeded messages. History is capped at 100, so every login in the experiments does 100 decrypt and verify operations. An empty room would make that step free and the experiment would only measure the network. The seeded messages are written the way the server writes a live one, so they verify in the same way.

6. Experiment 1, load balancer using only Sys2

The load balancer is started with one back end. The request count, the concurrency, the seeded room and the client machine are all the same as in Experiment 2.

```
$ ssh stu68_sys1 'bash ~/start-lb.sh "http://172.17.0.31:4000" 3000'
load balancer running on :3000 with 1 backends (sticky=true, TLS)
```

Figure 5. Setting up Experiment 1.

```
$ node chatload_main.js -url https://10.1.75.53:3229 -requests 600 -concurrency 30 -experiment chat-1-backend
{
  "experiment": "chat-1-backend",
  "requests": 600,
  "concurrency": 30,
  "successful": 600,
  "failed": 0,
  "throughput_rps": 8.73,
  "dropout_percent": 0,
  "p50_ms": 3405.939,
  "p95_ms": 4570.302,
  "p99_ms": 5035.758,
  "elapsed_seconds": 68.732
}
```

Figure 6. Chat workload. 600 sessions at concurrency 30, all served by Sys2.

```
$ ./client -url https://10.1.75.53:3229/health -requests 5000 -concurrency 40 -timeout 10s -insecure -
experiment http-1-backend
{
  "experiment": "http-1-backend",
  "requests": 5000,
  "concurrency": 40,
  "successful": 5000,
  "failed": 0,
  "throughput_rps": 725.7798984168378,
  "dropout_percent": 0,
  "p50_ms": 47.299,
  "p95_ms": 112.745,
  "p99_ms": 177.633,
  "elapsed_seconds": 6.889140924
}
```

Figure 7. HTTP workload. 5000 requests at concurrency 40. This is the median of three runs, because this workload varies a lot between runs.

7. Experiment 2, load balancer using all three back ends

```
$ ssh stu68_sys1 'bash ~/start-lb.sh "http://172.17.0.31:4000,http://172.17.0.32:4000,http://172.17.0.33:4000"
3000'
load balancer running on :3000 with 3 backends (sticky=true, TLS)
```

Figure 8. Same binary and same flags, with three back ends.

```
$ node chatload_main.js -url https://10.1.75.53:3229 -requests 600 -concurrency 30 -experiment chat-3-backends
{
  "experiment": "chat-3-backends",
  "requests": 600,
  "concurrency": 30,
  "successful": 600,
  "failed": 0,
  "throughput_rps": 23.17,
  "dropout_percent": 0,
  "p50_ms": 1005.477,
  "p95_ms": 2488.133,
  "p99_ms": 2951.901,
  "elapsed_seconds": 25.893
}
```

Figure 9. Chat workload. The same 600 sessions finished in 25.9 s instead of 68.7 s.

```
$ ./client -url https://10.1.75.53:3229/health -requests 5000 -concurrency 40 -timeout 10s -insecure -
experiment http-3-backends
{
  "experiment": "http-3-backends",
  "requests": 5000,
  "concurrency": 40,
  "successful": 5000,
  "failed": 0,
  "throughput_rps": 643.0598771017376,
  "dropout_percent": 0,
  "p50_ms": 65.002,
  "p95_ms": 164.845,
  "p99_ms": 290.241,
  "elapsed_seconds": 7.775325717
}
```

Figure 10. HTTP workload, median of three runs. Throughput did not improve. Section 9 explains why.

```

$ curl -sk https://10.1.75.53:3229/lb/metrics # after the chat run
{
  "failed": 0,
  "p50_ms": 1089.115,
  "p95_ms": 2716.702,
  "p99_ms": 3243.199,
  "per_backend": {
    "172.17.0.31:4000": 200,
    "172.17.0.32:4000": 200,
    "172.17.0.33:4000": 200
  },
  "success": 600,
  "total": 600
}

$ curl -sk https://10.1.75.53:3229/lb/metrics # after the HTTP run
{
  "failed": 0,
  "p50_ms": 15.929,
  "p95_ms": 99.103,
  "p99_ms": 190.442,
  "per_backend": {
    "172.17.0.31:4000": 1667,
    "172.17.0.32:4000": 1667,
    "172.17.0.33:4000": 1666
  },
  "success": 5000,
  "total": 5000
}

```

Figure 11. Round robin split 600 chat sessions as 200 each and 5000 HTTP requests as 1667, 1667 and 1666. Both are within one request of an even three way split.

8. Comparison table

CHAT WORKLOAD, 600 SESSIONS, CONCURRENCY 30									
EXPERIMENT	REQUESTS	SUCCESS	FAILED	THROUGHPUT	DROPOUT	P50	P95	P99	ELAPSED
1. Sys2 only	600	600	0	8.73 rps	0.0%	3405.9 ms	4570.3 ms	5035.8 ms	68.7 s
2. Sys2, Sys3 and Sys4	600	600	0	23.17 rps	0.0%	1005.5 ms	2488.1 ms	2951.9 ms	25.9 s
Change	-	-	-	2.7x faster	-	-70.5%	-45.6%	-41.4%	-62.3%

HTTP WORKLOAD, 5000 REQUESTS, CONCURRENCY 40, MEDIAN OF THREE RUNS									
EXPERIMENT	REQUESTS	SUCCESS	FAILED	THROUGHPUT	DROPOUT	P50	P95	P99	P50 AT LB
1. Sys2 only	5000	5000	0	725.8 rps	0.0%	47.3 ms	112.7 ms	177.6 ms	30.1 ms
2. Sys2, Sys3 and Sys4	5000	5000	0	643.1 rps	0.0%	65.0 ms	164.8 ms	290.2 ms	15.9 ms
Change	-	-	-	-11.4% within noise	-	+37.4%	+46.2%	+63.4%	-47.2%

9. Observations

The chat workload scales with the number of back ends. Three back ends served the same 600 sessions 2.7 times faster. The median latency dropped by 71% and the p99 tail dropped by 41%. Both runs finished with zero failures, so this is a result about capacity and latency and not about errors. The work here is CPU bound on the back ends. Each login decrypts and verifies 100 stored messages on a single vCPU. Spreading 30 concurrent clients over three machines gives each machine about a third of the work. The gap below a perfect 3x is the time spent in the load balancer and in the shared database, which both sit on Sys1 and are not replicated.

The HTTP workload does not separate the two setups. This is the expected result and not a broken measurement. A GET /health costs the back end almost nothing, so the back ends are not the limit. The limit is the single vCPU on Sys1, which the load balancer shares with mongod and which also handles TLS. Adding back ends does not raise that limit. Each setup was run three times to check this. One back end gave 611.9, 725.8 and 729.3 rps. Three back ends gave 643.1, 602.3 and 865.6 rps. The two ranges overlap completely, so the 11% gap between the medians in the table means nothing.

The back ends really were less loaded even so. The proof is on the load balancer side of the hop, where the measured p50 dropped from 30.1 ms to 15.9 ms. The back ends answered in half the time, and none of that reached the client. This is the point the comparison was meant to show. A load balancer only adds throughput when the back ends are the bottleneck. On a cheap request the proxy is the bottleneck instead, and extra machines add nothing.

The client side tail got worse on the HTTP run, rising from 113 to 165 ms at p95 and from 178 to 290 ms at p99. With three back ends the proxy keeps three connection pools instead of one and runs three health checks per interval instead of one, on a vCPU that was already full. This is a real cost of spreading out and it shows up only because this workload has no other bottleneck to hide it.

10. Health checks and failover

The health check loop polls every back end every 2 seconds and marks one down after three failures in a row. The test below stops the chat server on Sys3 while traffic is running, then starts it again.

```
### 1. all three backends healthy ###
http://172.17.0.31:4000    alive=True   served=0
http://172.17.0.32:4000    alive=True   served=0
http://172.17.0.33:4000    alive=True   served=0

### 2. six requests, all backends up, strict round robin ###
request 1 -> 200   served by 172.17.0.31:4000
request 2 -> 200   served by 172.17.0.32:4000
request 3 -> 200   served by 172.17.0.33:4000
request 4 -> 200   served by 172.17.0.31:4000
request 5 -> 200   served by 172.17.0.32:4000
request 6 -> 200   served by 172.17.0.33:4000

### 3. stop the chat server on Sys3 (172.17.0.32) ###
killed chat-2 (pid 20068) on Sys3
waiting 8s. The health check runs every 2s and needs 3 misses in a row.
http://172.17.0.31:4000    alive=True   served=2
http://172.17.0.32:4000    alive=False  served=2
http://172.17.0.33:4000    alive=True   served=2

### 4. six more requests with Sys3 down, no failures, Sys3 skipped ###
request 1 -> 200   served by 172.17.0.31:4000
request 2 -> 200   served by 172.17.0.33:4000
request 3 -> 200   served by 172.17.0.31:4000
request 4 -> 200   served by 172.17.0.33:4000
request 5 -> 200   served by 172.17.0.31:4000
request 6 -> 200   served by 172.17.0.33:4000

### 5. restart the chat server on Sys3 ###
UP after 1s
{"status":"ok","backend":"chat-2","uptimeSeconds":1,"onlineUsers":0}
waiting 6s...
http://172.17.0.31:4000    alive=True   served=5
http://172.17.0.32:4000    alive=True   served=2
http://172.17.0.33:4000    alive=True   served=5
```

```

### 6. six requests after recovery, Sys3 is back in the rotation ###
request 1 -> 200 served by 172.17.0.31:4000
request 2 -> 200 served by 172.17.0.32:4000
request 3 -> 200 served by 172.17.0.33:4000
request 4 -> 200 served by 172.17.0.31:4000
request 5 -> 200 served by 172.17.0.32:4000
request 6 -> 200 served by 172.17.0.33:4000

### 7. load balancer log ###
2026/08/24 15:41:06 http: TLS handshake error from 10.50.19.28:37334: remote error: tls: unknown certificate
2026/08/24 15:41:39 http://172.17.0.32:4000 marked down after 3 failed checks: Get "http://172.17.0.32:4000/health": dial
tcp 172.17.0.32:4000: connect: connection refused
2026/08/24 15:41:43 http://172.17.0.32:4000 is back up

```

Figure 12. A back end failing and recovering. Round robin narrows from three machines to two and then goes back to three. No client request failed while Sys3 was down.

11. The group chat application through the load balancer

The assignment asks for the group chat application to be reached through the load balancer. <https://10.1.75.53:3229> serves the chat application and every request to it is passed to one of the three copies on Sys2, Sys3 and Sys4.

Getting a real time application to work correctly behind a round robin proxy needed the three pieces described in Section 2. The test below checks all three against the live URL.

1. Two clients are put on different back ends and each one runs the real ECDSA login.
2. One sends a signed message and the other receives it. This only works if the broadcast crossed machines, which means the MongoDB adapter is working.
3. A third client on the remaining back end loads history from the shared database and gets the message back. It decrypts, and both the integrity tag and the signature verify. This shows the encryption key and the stored public keys are shared across all three copies.

```

ananya is pinned to backend 0 -> chat-1
vikram is pinned to backend 2 -> chat-3
ananya logged in
vikram logged in
ananya sent: "does this reach the other back end"
vikram received: "does this reach the other back end"
  message id      : 6a8c64f538dab9484ee91fc8
  signature check : valid
  stored ciphertext (first 32 chars): X5Yd046DE1hNbznXyKLIVyF5T33fd0sE...
meera is pinned to backend 1 -> chat-2
meera loaded 100 messages of history from the shared database
  ananya's message present : yes
  decrypted text           : "does this reach the other back end"
  integrity verdict        : ok
  signature verdict        : valid

RESULT: PASS. Chat works across all three back ends behind the load balancer.

```

Figure 13. The security properties of the group chat application still hold after moving to three copies behind the load balancer.

About the group deployment. The previous assignment was a group submission and its public URL runs on a separate container that this account has no access to. This report does not touch it. What is shown here is a separate working deployment of the same chat back end behind this assignment load balancer, on the four systems allocated to me.